

SIMTiX

A GPU-inspired SIMT accelerator on RISC-V

From First Principles to Silicon — A Learning Guide

Covers: the core idea • SIMT theory • build methodology • module-by-module function • full execution mechanism • how the result 964 is computed • a complete cycle-by-cycle trace

Krishna Gorai

github.com/Krishna-Gorai/simtix

Part 0 — The One-Sentence Idea

SIMTiX is a tiny GPU-style “do-the-same-thing-to-many-data-elements” accelerator, bolted onto an ordinary RISC-V CPU, where the CPU acts as the boss and the accelerator is the muscle for parallel work.

Everything else is detail about *how* the boss tells the muscle what to do, *how* the muscle does 8 things at once, and *how* you proved it works on a real FPGA.

Part 1 — The Problem It Solves (why GPUs exist)

A normal CPU executes **one instruction on one piece of data at a time**. To add two 8-element arrays:

```
for i in 0..7:  C[i] = A[i] + B[i]
```

a CPU loops 8 times — fetch/decode/execute the *same* add eight separate times. That is wasteful: the instruction is identical each iteration; only the *data* differs.

The GPU insight, called **SIMT** (Single Instruction, Multiple Threads): keep **one** instruction stream, but run it on **many parallel datapaths (“lanes”) at once**. One add instruction → 8 adds happen simultaneously, one per lane, each on its own data element. An 8-iteration loop becomes effectively one step. SIMTiX is a from-scratch hardware implementation of that idea, sized small (8 lanes, 4 warps) so it actually fits, builds, and runs end-to-end.

Part 2 — The Vocabulary (six words and the rest is easy)

Term	Meaning in SIMTiX	Where in code
Lane	One physical ALU+register slice. There are 8 (NUM_LANES=8). All lanes run the same instruction on different data.	warp_pool.sv per-lane loops
Thread	One logical worker doing the kernel for one data element (e.g. computing C[3]).	seeded via tid
Warp	A bundle of threads marching in lockstep on the 8 lanes. WARP_SIZE=8, so one warp = 8 threads = fills all 8 lanes.	wstate[warp]
Warp pool	The hardware holds 4 warps resident at once (NUM_WARPS=4) and interleaves them to hide memory latency.	warp_pool.sv
Kernel	The little program each thread runs: C[tid]=A[tid]+B[tid].	shared_mem.sv words 0x080..0x088
MMIO	Memory-Mapped I/O — the CPU controls the accelerator by writing special addresses, as if poking a device's registers.	mmio_regs.sv

Key relationship: 8 threads ÷ 8 lanes/warp = **1 warp** for the demo workload. So in the actual run, exactly one warp fills all 8 lanes; the multi-warp pool exists to scale but is not stressed by this tiny job.

Part 3 — The Methodology: how it was built

The project used a **green-milestone methodology**: every milestone is a self-contained step that lints clean, simulates green, and is committed, before the next one starts. The system is *always working* — each step makes it do more. This mirrors how real chip teams de-risk a design.

Milestone	What it added
M2	One warp, 8 lanes, basic RV32I execution across the lanes.
M3	The warp pool: 4 warps + round-robin scheduler (latency hiding).
M4	Memory coalescing: merge 8 lanes' accesses into one wide transaction.
M5	Control divergence (true SIMT): if/loops via a reconvergence stack.
M6	RV32M mul + per-warp scratchpad memory.
M7	Energy study: divergence-aware lane clock-gating accounting.
M8 / M9	Move the vector register file (VRF) and scratchpad into LUTRAM.
M10	The whole chip (chip_top): CPU + accelerator + shared mem + driver ROM.
M11	Full place & route → timing-closed bitstream on ZCU104 (xczu7ev).
M12	CPU-side input loading: the host CPU writes A/B at runtime.
M13	Post-implementation gate-level sim: the routed netlist still gives 964.

The lesson: correctness-first, incremental, always-green, always-committed.

Part 4 — System Anatomy (the map of chip_top)

The full chip is four blocks and two output pins (clk/rst in, done/result out):

```

+-----+-----+ chip_top.sv -----+
|
| clk --&gt;| +-----+ instr +-----+ | | | | |
| rst --&gt;| | riscv_pipeline|&lt;-----| cpu_driver_rom | |
| | (HOST CPU) | addr ---&gt;| (boss's program)| |
| | 5-stage | +-----+ |
| +-----+-----+ |
| | data bus (ALUResultM, WriteDataM, ReadDataM) |
| +-----v----- address decode (top 4 bits) -----+ |
| | 0x9.. -&gt; result reg 0x8.. -&gt; MMIO else-&gt;mem | |
| +-----+-----+-----+-----+ |
| | | | | | | | |
| +---v---+ +---v---+ +---v---+ |
| | result| | simt_accel |&lt;--&gt;| shared_mem | |
| | reg | | (ACCELERATOR)|imem| (kernel+A+B+C) | |
| +---+---+ | mmio_regs |dmem| 8 LUTRAM banks | |
| | | | warp_pool | +-----+ |
| | | +-----+ |
+-----+-----+-----+-----+
done &lt;--+
result &lt;--+

```

One-line job of each module:

Module	File	Job
riscv_pipeline	rtl/cpu/riscv_pipeline.v	The host CPU — an ordinary 5-stage RISC-V core. The “boss.”
cpu_driver_rom	rtl/soc/cpu_driver_rom.sv	The boss's program, baked into a ROM.
simt_accel	rtl/accel/simt_accel.sv	Accelerator shell: command registers + dispatcher FSM.

Module	File	Job
mmio_regs	rtl/accel/mmio_regs.sv	The control panel – registers the CPU writes to give commands.
warp_pool	rtl/accel/warp_pool.sv	The actual SIMT engine: lanes, warps, scheduler, memory engine, divergence stack.
shared_mem	rtl/soc/shared_mem.sv	Shared memory holding the kernel + A/B/C arrays. CPU and accelerator both reach it.
simtix_pkg	rtl/accel/simtix_pkg.sv	Constants – sizes, the MMIO register map, opcode encodings.

Part 5 — The Execution Mechanism, End to End

The theory of how it actually runs, as a story in six acts — what happens from the moment you press reset.

Act 1 — Boot & the CPU loads the inputs (M12)

On reset the CPU's PC is 0x0000_0000 and it fetches from `cpu_driver_rom`. The first 14 instructions are a **store loop** that writes the inputs into shared memory:

```
A[i] = 10 + i    @ 0x300    -&gt;  A = {10,11,12,13,14,15,16,17}
B[i] = 100 + 2i @ 0x340    -&gt;  B = {100,102,104,106,108,110,112,114}
```

Address 0x300 has top nibble 0x3 — neither 0x8 nor 0x9 — so the decoder routes it to shared memory. The memory no longer preloads A/B, so a correct result **proves the CPU genuinely moved the data**.

Act 2 — The CPU programs the accelerator over MMIO

The CPU loads `x1 = 0x8000_0000` and stores the command block. Top nibble 0x8 asserts `is_mmio`, so these stores land in `mmio_regs`:

```
kernel_pc = 0x200    base_a = 0x300    base_b = 0x340
base_c     = 0x380    N           = 8
```

Each store hits the `case(offset)` in `mmio_regs` and latches one register. This is the **driver/device contract**: software fills known registers, then rings a doorbell.

Act 3 — The doorbell: GO

Writing 1 to `REG_CTRL` (offset 0x14) produces a **1-cycle go_pulse**, which drives the dispatcher FSM in `simt_accel`: `D_IDLE` → `D_LAUNCH` (pulses `pool_start`, handing the whole grid to the warp pool) → `D_RUN` (counts cycles until `pool_done`) → `D_DONE` (latches `REG_CYCLES`).

Act 4 — The warp pool executes the kernel (the SIMT core)

On start the pool computes `total_warps = ceil(8/8) = 1`, so **one warp** spawns and fills all 8 lanes. Each lane's `a0` is seeded with its thread id (lane 0 → tid 0 ... lane 7 → tid 7); the kernel's argument registers (`a1=&A`, `a2=&B`, `a3=&C`, `a4=N`) are seeded identically. The warp then runs the 9-instruction kernel one instruction at a time, but **8 lanes wide**:

```

slli t0, a0, 2      # t0 = tid*4    (byte offset)    &lt;- each lane: its own tid
add  t1, a1, t0     # t1 = &A[tid]
lw   t2, 0(t1)      # t2 = A[tid]                  &lt;- 8 loads at once
add  t3, a2, t0     # t3 = &B[tid]
lw   t4, 0(t3)      # t4 = B[tid]                  &lt;- 8 loads at once
add  t2, t2, t4      # t2 = A[tid] + B[tid]          &lt;- 8 adds at once
add  t5, a3, t0     # t5 = &C[tid]
sw   t2, 0(t5)      # C[tid] = result                &lt;- 8 stores at once
ecall               # thread done

```

The execution loop does up to three things each cycle: **spawn** a warp into a free slot, **issue** one runnable warp on the shared datapath (round-robin), and **advance the memory engine**. For a compute op the hardware computes the result independently for all 8 lanes in parallel — one instruction, eight results. That is SIMT in silicon.

Act 5 — Memory coalescing (the M4 cleverness)

For `lw t2, 0(t1)` the 8 lanes generate addresses 0x300, 0x304, ... 0x31C — all inside one 32-byte cache line. The coalescing engine services **all 8 lanes in a single 256-bit transaction** instead of 8. `dbg_mem_txns` counts them; the whole kernel uses just **3** (one per `lw/lw/sw`). The wide line comes from `shared_mem`'s 8 LUTRAM banks read in parallel.

Act 6 — Retire, read-back, publish

Each lane hits `ecall` and the warp retires; when nothing is left the pool pulses done. The CPU has been **polling** `STATUS.DONE`; once set it reads `C[0..7]` back, sums them, and stores the sum to 0x9000_0000 (top nibble 0x9 → result register latches the value and raises the chip's done pin). Then it halts in a self-loop.

Part 6 — How the Result 964 Is Computed

```

A[i] = 10 + i      -> 10, 11, 12, 13, 14, 15, 16, 17
B[i] = 100 + 2i    -> 100,102,104,106,108,110,112,114
-----
C[i] = A[i] + B[i] -> 110,113,116,119,122,125,128,131 = 110 + 3i

result = sum C[i] for i=0..7
        = sum (110 + 3i)
        = (8 x 110) + 3 x (0+1+2+3+4+5+6+7)
        = 880 + 3 x 28
        = 880 + 84
        = 964

```

Why a sum and not the array? A single 32-bit result pin cannot show 8 values, so the CPU collapses them into one **checksum**. 964 is reachable only if every stage worked: the CPU wrote A/B (M12), MMIO reached the accelerator, the warp ran the kernel on all lanes, coalesced memory hit the right addresses, and the CPU read C back and summed it. Any broken link gives a different number ($A=B=0 \rightarrow 0$; a missed C write leaves 0xdeadbeef poison). **964 is a single-number proof that the entire CPU → memory → accelerator → memory → CPU loop is correct.**

Part 7 — The Advanced Bits (for interviews)

1. The reconvergence stack (control divergence, M5). Real GPU code has `if (cond)` where some lanes branch and some do not. A per-warp SIMT stack pushes a frame on a

divergent branch so one lane group runs while the other waits, then they reconverge at a join PC. `dbg_divergences` counts events. (The demo kernel is uniform, but the hardware handles divergence.)

2. Round-robin scheduling & latency hiding (M3). With 4 warp slots and one datapath, the scheduler issues a different warp each cycle. When one warp stalls on memory, others keep computing — the single most important GPU performance idea.

3. The VRF write arbiter (M8/M9). The register file is LUTRAM with one write port per lane. When a compute result and a returning load collide, the memory write wins and the compute instruction re-issues next cycle (it is idempotent). The `reg_written` trick avoids clearing 32 registers on spawn (which broke RAM inference): an unwritten register simply returns its seed value on read.

4. The scratchpad (M6). A small per-warp on-chip memory at `0x4000_0000`, served in-engine and never touching global memory — the GPU “shared memory” concept.

Part 8 — Four Independent Proofs (all give 964)

Level	What it proves	How
1. RTL simulation	The logic is correct	Verilator/xsim on the .sv source; <code>tb_chip_top</code> checks <code>result==964</code>
2. GUI behavioral sim	Same, in Vivado's simulator	<code>vivado_project</code> → Run Simulation → 964
3. Synth + impl + bitstream (M11)	Maps to real gates, closes timing at 100 MHz, emits <code>chip_top.bit</code>	<code>impl_chip.tcl</code> → place & route → <code>write_bitstream</code>
4. Post-impl gate sim (M13)	The actual routed netlist (real gates) still gives 964	<code>write_verilog -mode funcsim</code> → xsim with <code>glbl</code>

Level 4 is the strongest: it simulates the *physical* design Vivado produced. 964 at the gate level means the silicon would do this.

Part 9 — The Honest Scale Caveat

The demo workload is deliberately small: 8 threads = exactly 1 warp, no divergence, perfectly contiguous memory. So this run does not *stress* the multi-warp scheduler, divergence stack, or scattered coalescing — those are exercised by the per-milestone testbenches. `chip_top` exists to prove the **full integration** end-to-end on real silicon, with a result simple enough to hand-verify. Both stories are true and worth telling: “the chip integrates and runs on FPGA” and “the engine implements full SIMT with divergence and coalescing.”

Part 10 — Cycle-by-Cycle Trace

We follow one warp through the 9-instruction kernel, cycle by cycle, watching all 8 lanes move in lockstep and the memory engine fire. Traced directly from the `always_ff` at `warp_pool.sv`.

Setup — what is true before instruction 0

When the dispatcher pulses `start`, the pool spawns one warp (`total_warps = ceil(8/8) = 1`). All 8 lanes get seeded registers; the only difference between lanes is **a0 = tid = lane index**. This is the whole trick of SIMT in one table:

Register	Seed	Lane 0	Lane 7	Meaning
a0 (x10)	tid	0	7	the thread id – differs per lane
a1 (x11)	base_a	0x300	0x300	&A (same for all)
a2 (x12)	base_b	0x340	0x340	&B
a3 (x13)	base_c	0x380	0x380	&C
a4 (x14)	N	8	8	thread count

The kernel, with PCs and register targets

(t0=x5, t1=x6, t2=x7, t3=x28, t4=x29, t5=x30)

PC	word	instruction	per-lane effect (lane l)
0x200	0x80	slli t0, a0, 2	t0 = tid*4 = 4l (byte offset)
0x204	0x81	add t1, a1, t0	t1 = 0x300 + 4l = &A[l]
0x208	0x82	lw t2, 0(t1)	t2 = A[l] <- MEMORY
0x20C	0x83	add t3, a2, t0	t3 = 0x340 + 4l = &B[l]
0x210	0x84	lw t4, 0(t3)	t4 = B[l] <- MEMORY
0x214	0x85	add t2, t2, t4	t2 = A[l] + B[l]
0x218	0x86	add t5, a3, t0	t5 = 0x380 + 4l = &C[l]
0x21C	0x87	sw t2, 0(t5)	C[l] = t2 <- MEMORY
0x220	0x88	ecall	retire thread

The master timeline

Three things can happen each cycle (spawn / issue / memory-service). C0 = the cycle start is high. Watch lane 0 vs lane 7 to see SIMT, and the W_MEM rows to see a warp park itself on memory.

Cyc	State (during)	PC	Instr issued	Lane 0	Lane 7	Mem engine	End-of-cycle effect
C0	—	—	—	—	—	idle	latch grid; total_warps=1; all slots EMPTY; running=1
C1	EMPTY→spawn	—	(none — warp not RUN yet)	—	—	idle	warp0 → W_RUN, seeded, PC=0x200, mask=0xFF
C2	W_RUN	0x200	slli t0,a0,2	t0=0	t0=28	idle	t0=4l written all lanes; PC→0x204
C3	W_RUN	0x204	add t1,a1,t0	t1=0x300	t1=0x31C	idle	&A[l] in t1; PC→0x208
C4	W_RUN	0x208	lw t2,0(t1)	req 0x300	req 0x31C	LATCH	warp→W_MEM; mem_pending=0xFF; PC parked 0x20C
C5	W_MEM	(idle)	(none)	t2←10	t2←17	1 txn ✓	8 lanes share line 0x300..0x31F → 1 coalesced read; A[l]→t2; warp→W_RUN; PC→0x20C
C6	W_RUN	0x20C	add t3,a2,t0	t3=0x340	t3=0x35C	idle	&B[l] in t3; PC→0x210
C7	W_RUN	0x210	lw t4,0(t3)	req 0x340	req 0x35C	LATCH	warp→W_MEM; PC parked 0x214
C8	W_MEM	(idle)	(none)	t4←100	t4←114	1 txn ✓	line 0x340..0x35F → 1 coalesced read; B[l]→t4; warp→W_RUN; PC→0x214
C9	W_RUN	0x214	add t2,t2,t4	t2=110	t2=131	idle	t2 = A[l]+B[l] = C[l] = 110+3l; PC→0x218
C10	W_RUN	0x218	add t5,a3,t0	t5=0x380	t5=0x39C	idle	&C[l] in t5; PC→0x21C
C11	W_RUN	0x21C	sw t2,0(t5)	st 110→0x380	st 131→0x39C	LATCH	warp→W_MEM; per-lane store data latched; PC parked 0x220
C12	W_MEM	(idle)	(none)	—	—	1 txn ✓	8 lanes merged → 1 coalesced line WRITE; C[0..7] land in shared_mem; warp→W_RUN; PC→0x220
C13	W_RUN	0x220	ecall	retire	retire	idle	warp0 → W_DONE
C14	W_DONE	—	(none)	—	—	idle	nothing busy → running=0, done<=1
C15	—	—	—	—	—	—	done pulses → dispatcher latches REG_CYCLES≈16

By C12 the 8 results {110,113,116,119,122,125,128,131} sit in shared memory. The CPU then reads them back and sums to 964.

Zoom in: how ONE memory instruction works (C4 → C5)

A `lw` is not a single-cycle event; it splits into **issue** and **service**.

C4 — Issue (the warp hands off and steps aside). The issue logic sees `is_mem && !mem_busy` and latches the request:

```
mem_busy      &lt;= 1
mem_pending   &lt;= cur_mask           // 0xFF - all 8 lanes want data
mem_addr_lane[l]&lt;= addr[l]           // 0x300, 0x304, ... 0x31C
mem_rd        &lt;= t2
mem_next_pc   &lt;= 0x20C               // where to resume after the load
wstate[0]     &lt;= W_MEM               // &lt;- warp parks itself
```

The warp takes itself out of `W_RUN` and will not be fetched again until the memory engine wakes it. No load value is produced this cycle — only the request.

C5 — Service + coalesce + writeback. With `mem_busy=1`, the memory engine: (1) picks the leader = lowest pending lane (lane 0, `addr 0x300`); (2) coalesces every pending lane sharing that line tag — all 8 are inside `0x300..0x31F`, so `grp=0xFF`; (3) drives one 256-bit line read; (4) slices each lane's word out and writes it to `t2` (lane 0 gets 10, lane 7 gets 17); (5) sees all lanes serviced, so `mem_busy<=0`, `PC<=0x20C`, `warp→W_RUN`. **8 loads → 1 transaction.**

The “bubble” — and why the warp pool exists

In cycles C5, C8, C12 the issue datapath fetches nothing (the only warp is parked in `W_MEM`). These are **stall bubbles**. With 4 warps (e.g. `N=32`) the round-robin scheduler would issue a different warp's compute during those cycles:

```
C4: warp 0 issues lw      -&gt; warp 0 parks in W_MEM
C5: warp 0's load services AND warp 1 issues its slli  &lt;- bubble filled!
C6: warp 1 issues add     AND (warp 0 back in W_RUN)
...
```

Memory latency disappears behind other warps' useful work — the single most important idea in GPU performance, already present in `warp_pool`. The demo simply lacks enough threads to show it off.

The cycle budget

```
5 compute instrs x 1 cycle each      = 5    (slli, add, add, add, add)
3 memory instrs  x 2 cycles each      = 6    (issue + service: lw, lw, sw)
1 ecall          x 1 cycle            = 1
-----
kernel execution                                = 12 cycles  (C2 -&gt; C13)
+ launch / spawn overhead (C0, C1)             = 2
+ completion + done pulse (C14, C15)           = 2
-----
total ~ 16 cycles -&gt; this is REG_CYCLES, readable by the CPU
```

Each memory op costs one extra bubble cycle *here* — but that is exactly the cost a full warp pool would hide. Both readings are true: “memory ops cost 2 cycles on a single warp” and “with a full pool those service cycles are free.”

What you can now see in the silicon

- **Lockstep with a twist:** all 8 lanes run the same instruction each cycle; only `tid` (seeded in `a0`) makes results differ.

- **Memory is a two-phase background operation:** issue (park the warp) then service (coalesce + write back + resume).
- **Coalescing is real and measurable:** 8 contiguous accesses → 1 transaction; dbg_mem_txns ends at 3.
- **Bubbles reveal the warp pool's purpose:** the empty issue slots in C5/C8/C12 are exactly what multi-warp scheduling fills.
- **The result exists in memory by C12,** before the CPU reads it back — the 964 sum is the CPU's verification of the accelerator's work.

Part 11 — Bonus: A Divergent-Kernel Trace (the reconvergence stack)

The vector-add kernel is *uniform* — every lane takes the same path, so the SIMT stack never grows. To see Part 7's reconvergence stack actually work, here is an illustrative kernel with a data-dependent branch. (This is a teaching example, not the kernel in the ROM; the divergence hardware it exercises is the real M5 logic in warp_pool.sv.)

The divergent kernel

Single-sided if (the M5-supported form): lanes with tid < 4 run the body; the rest skip it. The guard branch is the negation — it is *taken* (skips the body) when tid ≥ 4, so the lanes that run the body are the fall-through lanes.

PC	instruction	per-lane effect	
0x200	addi t0, x0, 4	t0 = 4	(uniform)
0x204	bge a0, t0, 0x210	if tid≥4 -> JOIN (skip body)	DIVERGENT (forward if)
0x208	addi t1, a0, 100	t1 = tid + 100	(body: lanes tid<4)
0x20C	ori t1, t1, 0	t1 = t1	(body 2nd instr)
0x210	add t2, a0, t1	JOIN: t2 = tid + t1	(reconverged, all lanes)
0x214	ecall	retire	

Lanes that skip the body never write t1, so it keeps its spawn seed 0. The result therefore differs sharply between the two halves of the warp:

Lane (tid)	Path	t1	t2 = tid + t1
0–3 (tid<4)	runs body	tid+100 -> 100,101,102,103	2*tid+100 -> 100,102,104,106
4–7 (tid≥4)	skips body	0 (spawn seed)	tid -> 4,5,6,7

The SIMT stack, in one breath

Each warp owns a small stack of frames. The top-of-stack (TOS) frame defines the warp's live **PC**, its **active mask** (which lanes execute), and an **RPC** (the join PC at which this frame pops). A divergent branch pushes a frame; when a pushed frame's PC reaches its RPC it pops, and the warp reconverges into the frame below (which already holds the full union mask).

After spawn (C1):	sp=0	TOS ->[pc=0x200 mask=0xFF rpc=BOTTOM]	
After divergent br (C3):	sp=1	TOS ->[pc=0x208 mask=0x0F rpc=0x210]	body lanes (tid<4)
		[pc=0x210 mask=0xFF rpc=BOTTOM]	waiting at join
After pop (C6):	sp=0	TOS ->[pc=0x210 mask=0xFF rpc=BOTTOM]	RECONVERGED

The divergent timeline

Watch the Active mask column narrow to the body lanes after the branch (C3), then widen back to all 8 after the pop (C6). bge funct3 = 101. (C0 launch is omitted; C1 = spawn, as before.)

Cyc	sp	PC	Instr	Active mask	Per-lane effect	Stack action / end-of-cycle
C1	0	—	(spawn)	0xFF (all 8)	warp0 seeded; a0 = tid per lane	stack init: frame0{pc=0x200, mask=0xFF, rpc=BOTTOM}
C2	0	0x200	addi t0,x0,4	0xFF (all 8)	t0 = 4 (all lanes)	uniform compute; TOS.pc→0x204
C3	0	0x204	bge a0,t0,0x210	0xFF (all 8)	cond tid>=4: lanes 4-7 taken, lanes 0-3 fall-through	DIVERGENT (fwd if): dbg_divergences=1. TOS.pc←0x210 (JOIN), mask←0xFF; PUSH frame{pc=0x208, mask=0x0F, rpc=0x210}; sp→1
C4	1	0x208	addi t1,a0,100	0x0F (lanes 0-3)	lanes 0-3: t1 = tid+100 (100..103); lanes 4-7: masked, no write	body; TOS.pc→0x20C
C5	1	0x20C	ori t1,t1,0	0x0F (lanes 0-3)	lanes 0-3: t1 unchanged (100..103); lanes 4-7: idle (masked)	body; TOS.pc→0x210 (= RPC)
C6	1	0x210	(none – POP)	—	— (no datapath work)	cur_pc == RPC → POP; sp→0; reconverge into frame0 (mask 0xFF)
C7	0	0x210	add t2,a0,t1	0xFF (all 8)	lanes 0-3: t2 = 2*tid+100 (100,102,104,106); lanes 4-7: t2 = tid (4,5,6,7)	reconverged compute; TOS.pc→0x214
C8	0	0x214	ecall	0xFF (all 8)	all lanes retire	warp0 → W_DONE
C9	—	—	(none)	—	—	nothing busy → done<=1

Active mask narrows 0xFF → 0x0F at the divergent branch (C3) and widens back to 0xFF at the pop (C6). dbg_divergences ends at 1.

The divergence penalty

During C4 and C5 only 4 of the 8 lanes do useful work — the other 4 are masked off and idle. That is **8 wasted lane-cycles**: the fundamental SIMT divergence penalty (lane underutilization). The M7 counters measure exactly this — `dbg_issued_insns` counts datapath instructions (each could clock all 8 lanes) while `dbg_active_lanes` sums the lanes actually active; their ratio is lane utilization. A general two-sided if/else would be worse: it serializes **both** arms (the else lanes run in extra cycles too), which is why the M5 convention favours single-sided ifs written as two of them.

Key takeaways

- **A divergent branch pushes a stack frame** and narrows the active mask to one lane group; the other group is parked in the frame below, waiting at the join.
- **Masked-off lanes do nothing** — their writeback is gated by the active mask, so they neither corrupt state nor make progress.
- **A pushed frame pops when its PC reaches its RPC** (the join), restoring the full union mask — that is reconvergence.
- **Nesting works** up to the stack depth (`SDEPTH = 8`): each further divergence pushes another frame.
- **dbg_divergences = 1** for this kernel — exactly one divergent-branch event.

Appendix — File Map

File	Role
<code>rtl/soc/chip_top.sv</code>	Top-level SoC: CPU + accel + memory + result register.
<code>rtl/soc/cpu_driver_rom.sv</code>	Host driver program (store loop, MMIO, poll, readback).
<code>rtl/soc/shared_mem.sv</code>	8-bank LUTRAM shared memory (kernel + A/B/C).
<code>rtl/accel/simt_accel.sv</code>	Accelerator shell: <code>mmio_regs</code> + <code>warp_pool</code> + dispatcher FSM.
<code>rtl/accel/mmio_regs.sv</code>	Memory-mapped command/status registers.
<code>rtl/accel/warp_pool.sv</code>	The SIMT engine (lanes, warps, scheduler, mem engine, stack).
<code>rtl/accel/simtix_pkg.sv</code>	Global parameters, MMIO map, ISA constants.
<code>rtl/cpu/riscv_pipeline.v</code>	Reused 5-stage RISC-V host core.
<code>tests/tb_chip_top.sv</code>	Self-checking testbench (expects <code>result==964</code>).
<code>fpga/impl_chip.tcl</code>	Synthesis → place & route → bitstream flow.
<code>fpga/create_project.tcl</code>	Builds a clickable Vivado GUI project.

<i>Generated for personal study of the SIMTiX project. All facts are traced to the RTL in the repository.</i>